

HỆ ĐIỀU HÀNH

(OPERATING SYSTEMS)

ĐỀ CƯƠNG ÔN TẬP TOÀN DIỆN ĐI THI

Trọng tâm ôn tập:

Chương 5: Đồng bộ tiến trình (Process Synchronization)

Chương 7: Quản lý bộ nhớ (Memory Management)

Chương 8: Bộ nhớ ảo (Virtual Memory)

Tài liệu tổng hợp chuẩn xác, đầy đủ ý, in đậm từ khóa cốt lõi
Kèm hướng dẫn giải chi tiết các dạng bài tập tính toán trọng tâm

Mục lục

1	CHƯƠNG 5: ĐỒNG BỘ TIẾN TRÌNH (PROCESS SYNCHRONIZATION)	3
1.1	Race Condition và Vùng tranh chấp (Critical Section)	3
1.2	Ba yêu cầu bắt buộc đối với lời giải cho Critical Section	3
1.3	Các giải pháp phần mềm và Giải thuật Peterson	4
1.3.1	1. Giải pháp vô hiệu hóa ngắt (Disable Interrupts)	4
1.3.2	2. Giải pháp phần mềm 1 (Dùng biến turn)	4
1.3.3	3. Giải pháp phần mềm 2 (Dùng mảng flag[])	4
1.3.4	4. Giải thuật Peterson (Hoàn chỉnh cho 2 tiến trình)	4
1.4	Các giải pháp hỗ trợ từ phần cứng	4
1.5	Khóa Mutex (Mutex Locks)	5
1.6	Semaphore (Đèn hiệu)	5
1.6.1	1. Phân loại Semaphore	5
1.6.2	2. Hiện thực Semaphore không Busy Waiting	6
1.6.3	3. Ba ứng dụng kinh điển của Semaphore	6
1.7	Monitor và Biến điều kiện (Condition Variables)	6
1.8	Vấn đề Liveness	7
1.9	Ba bài toán đồng bộ kinh điển	7
1.9.1	1. Bài toán Bounded-Buffer (Producer - Consumer)	7
1.9.2	2. Bài toán Readers - Writers (Biến thể 1: Ưu tiên Readers)	7
1.9.3	3. Bài toán Dining-Philosophers (Triết gia ăn tối)	8
2	CHƯƠNG 7: QUẢN LÝ BỘ NHỚ (MEMORY MANAGEMENT)	9
2.1	Khái niệm cơ sở và Các kiểu địa chỉ nhớ	9
2.2	Chuyển đổi địa chỉ (Address Binding) và Nạp/Liên kết động	9
2.2.1	1. Ba thời điểm kết gán địa chỉ (Address Binding)	9
2.2.2	2. Nạp động (Dynamic Loading) và Liên kết động (Dynamic Linking)	9
2.3	Các mô hình phân vùng bộ nhớ và Hiện tượng phân mảnh	10
2.4	Cơ chế Phân trang (Paging) và Bảng trang	10
2.5	TLB và Thời gian truy xuất hiệu dụng (EAT)	10
2.6	Bảo vệ bộ nhớ và Cơ chế Hoán vị (Swapping)	11
3	CHƯƠNG 8: BỘ NHỚ ẢO (VIRTUAL MEMORY)	12
3.1	Tổng quan về Bộ nhớ ảo và Demand Paging	12
3.2	Quy trình xử lý lỗi trang (Page-Fault Service Routine - PFSR)	12
3.3	Các thuật toán thay thế trang (Page Replacement Algorithms)	12
3.3.1	1. Thuật toán FIFO (First-In-First-Out)	12
3.3.2	2. Thuật toán Tối ưu (OPT - Optimal)	13
3.3.3	3. Thuật toán LRU (Least Recently Used)	13
3.4	Cấp phát khung trang (Frame Allocation)	13
3.5	Hiện tượng Trì trệ (Thrashing) và Mô hình Working-Set	13
4	CÁC DẠNG BÀI TẬP TÍNH TOÁN ĐI THI (HƯỚNG DẪN CHI TIẾT)	15
4.1	Dạng 1: Cấp phát bộ nhớ động (First-fit, Best-fit, Worst-fit, Next-fit)	15
4.2	Dạng 2: Phân trang và Tính toán số bit địa chỉ	15
4.3	Dạng 3: Tính thời gian truy xuất hiệu dụng (EAT với TLB)	16
4.4	Dạng 4: Mô phỏng thuật toán thay trang (FIFO, OPT, LRU)	16
4.5	Dạng 5: Cấp phát khung trang theo tỉ lệ (Proportional Allocation)	20

5	BẢNG SO SÁNH TỔNG HỢP VÀ BỘ CÂU HỎI PHẢN XẠ NHANH	21
5.1	Bảng phân biệt các khái niệm then chốt	21
5.2	Bộ câu hỏi phản xạ nhanh lý thuyết đi thi	21

1. CHƯƠNG 5: ĐỒNG BỘ TIẾN TRÌNH (PROCESS SYNCHRONIZATION)

1.1. Race Condition và Vùng tranh chấp (Critical Section)

KHÁI NIỆM CỐT LÕI: Race Condition (Nghịch cảnh đua tranh)

Race Condition là hiện tượng xảy ra khi nhiều tiến trình (hoặc luồng) cùng truy cập và thao tác đồng thời trên dữ liệu chia sẻ. Kết quả thực thi cuối cùng phụ thuộc vào thứ tự đan xen lệnh của các tiến trình đó, dẫn đến nguy cơ dữ liệu bị sai lệch và không nhất quán (inconsistency).

Nguyên nhân từ mức máy (Ví dụ bài toán Producer - Consumer):

Lệnh tăng và giảm ở ngôn ngữ bậc cao thực chất gồm 3 chỉ thị ở mức hợp ngữ:

- `count++` → `reg1 = count; reg1 = reg1 + 1; count = reg1;`
- `count--` → `reg2 = count; reg2 = reg2 - 1; count = reg2;`

Khi hết quantum time và chuyển ngữ cảnh giữa chừng, các chỉ thị này đan xen nhau khiến giá trị `count` cuối cùng bị sai.

KHÁI NIỆM CỐT LÕI: Critical Section (Vùng tranh chấp)

Critical Section (CS) là đoạn mã lệnh mà ở đó tiến trình thao tác trên dữ liệu dùng chung (ghi biên, cập nhật bảng, ghi tệp tin). Để bảo vệ dữ liệu, tại một thời điểm chỉ cho phép duy nhất một tiến trình thực thi trong CS của nó.

Mô hình chuẩn của tiến trình truy cập CS:

```
1 while (true) {  
2     entry section    // Xin quyền truy cập vào CS  
3     critical section // Thao tác trên dữ liệu dùng chung  
4     exit section     // Thông báo đã rời CS, mở khóa cho tiến trình khác  
5     remainder section // Các mã lệnh còn lại không tranh chấp  
6 }
```

1.2. Ba yêu cầu bắt buộc đối với lời giải cho Critical Section

LƯU Ý QUAN TRỌNG ĐI THI: 3 YÊU CẦU BẮT BUỘC (RẤT HAY HỎI THI)

Mọi giải pháp giải quyết bài toán vùng tranh chấp bắt buộc phải thỏa mãn đồng thời 3 điều kiện:

1. **Loại trừ tương hỗ (Mutual Exclusion):** Khi một tiến trình đang thực thi trong Critical Section của nó thì không có bất kỳ tiến trình nào khác được thực thi trong CS của chúng.
2. **Tiến triển (Progress):** Nếu không có tiến trình nào trong CS và có các tiến trình muốn vào CS, thì chỉ những tiến trình không ở trong remainder section mới được tham gia quyết định tiến trình tiếp theo được vào. Tiến trình bên ngoài không được cản trở tiến trình khác vào CS (ngăn ngừa **Deadlock**).
3. **Chờ đợi có giới hạn (Bounded Waiting):** Phải có một giới hạn xác định về số lần các tiến trình khác được phép vào CS sau khi một tiến trình đã yêu cầu vào và trước khi yêu cầu đó được chấp thuận (ngăn ngừa **Starvation - đói tài nguyên**).

1.3. Các giải pháp phần mềm và Giải thuật Peterson

1.3.1. 1. Giải pháp vô hiệu hóa ngắt (Disable Interrupts)

- **Cơ chế:** entry section gọi tắt ngắt; exit section bật lại ngắt.
- **Nhược điểm:** Nguy hiểm nếu CS lặp vô tận (hệ thống bị tê liệt). **Không hoạt động trên hệ thống đa CPU (Multiprocessor)** vì việc tắt ngắt trên một CPU không thể cấm các CPU khác truy cập bộ nhớ chung.

1.3.2. 2. Giải pháp phần mềm 1 (Dùng biến turn)

- **Cơ chế:** Chia sẻ biến `int turn;`. Tiến trình P_i chờ `while (turn == j);`, sau CS gán `turn = i;`.
- **Đánh giá:** Thỏa Mutual Exclusion nhưng **vi phạm Progress và Bounded Waiting** do ép buộc luân phiên gượng ép (*strict alternation*). Nếu P_j ở remainder section hoặc kết thúc, P_i sẽ bị chặn vĩnh viễn dù CS đang rảnh.

1.3.3. 3. Giải pháp phần mềm 2 (Dùng mảng flag[])

- **Cơ chế:** `boolean flag[2];`. Tiến trình P_i đặt `flag[i] = true;`, sau đó kiểm tra `while (flag[j]);`. Rồi CS đặt `flag[i] = false;`.
- **Đánh giá:** Thỏa Mutual Exclusion nhưng **vi phạm Progress** vì nếu cả hai cùng đặt cờ flag lên true đồng thời, cả hai sẽ cùng chờ nhau vô hạn (**Deadlock**).

1.3.4. 4. Giải thuật Peterson (Hoàn chỉnh cho 2 tiến trình)

Kết hợp cả 2 biến `flag[2]` và `turn`:

```

1 // Ma nguon tien trinh P_i (tien trinh doi thu la P_j)
2 while (true) {
3     flag[i] = true;                // 1. P_i san sang vao CS
4     turn = j;                     // 2. Nhuong luot cho P_j
5     while (flag[j] && turn == j); // 3. Cho neu P_j muon vao VA dang la luot cua
    P_j
6
7     /* CRITICAL SECTION */
8
9     flag[i] = false;              // 4. Roi CS, ha co
10
11    /* REMAINDER SECTION */
12 }
```

Đánh giá giải thuật Peterson:

- Đảm bảo thỏa mãn cả 3 điều kiện: Mutual Exclusion, Progress và Bounded Waiting.
- **Hạn chế trên kiến trúc máy tính hiện đại:** Các CPU hiện đại và trình biên dịch có cơ chế tối ưu sắp xếp lại thứ tự chỉ thị (**Instruction Reordering**) đối với các thao tác độc lập (đảo thứ tự gán `flag[i]` và `turn`), dẫn đến giải thuật bị sai lệch.
- **Khắc phục:** Sử dụng rào chắn bộ nhớ (**Memory Barrier**).

1.4. Các giải pháp hỗ trợ từ phần cứng

1. **Memory Barrier:** Chỉ thị phần cứng ép buộc mọi thao tác đọc/ghi bộ nhớ trước nó phải được hoàn tất và hiển thị cho toàn bộ hệ thống CPU trước khi các lệnh sau nó được thực hiện.
2. **Lệnh test_and_set:** Thao tác phần cứng **đơn nguyên (atomic)** đọc giá trị cũ và ghi đè giá trị mới là true:

```

1 boolean test_and_set(boolean *target) {
2     boolean rv = *target;
3     *target = true;
4     return rv;
5 }
6 // Áp dụng: while (test_and_set(&lock)); /* CS */ lock = false;

```

3. **Lệnh compare_and_swap (CAS):** Thao tác đơn nguyên so sánh giá trị bộ nhớ với giá trị kỳ vọng, chỉ cập nhật nếu bằng nhau:

```

1 int compare_and_swap(int *value, int expected, int new_value) {
2     int temp = *value;
3     if (*value == expected) *value = new_value;
4     return temp;
5 }
6 // Áp dụng: while (compare_and_swap(&lock, 0, 1) != 0); /* CS */ lock = 0;

```

4. **Biến đơn nguyên (Atomic Variables):** Hỗ trợ thực hiện các phép toán cơ bản (như cộng, trừ) trên kiểu dữ liệu số mà không bị ngắt quãng.

1.5. Khóa Mutex (Mutex Locks)

Khóa Mutex (Mutual Exclusion Lock) là công cụ đồng bộ cấp cao đơn giản nhất. Tiến trình gọi `acquire()` để lấy khóa trước khi vào CS và `release()` để nhả khóa khi ra khỏi CS.

Đặc điểm	Spinlock (Busy Waiting)	Mutex Sleep - Wakeup (Non-busy waiting)
Cơ chế chờ	Lặp liên tục kiểm tra biến cờ (<code>while (!available);</code>).	Gọi <code>block()</code> đưa tiến trình vào hàng đợi ngủ ; khi mở khóa gọi <code>wakeup()</code> đánh thức.
Sử dụng CPU	Lãng phí chu kỳ CPU do bận lặp.	Tiết kiệm CPU , nhường CPU cho tiến trình khác.
Chi phí chuyển ngữ cảnh	Không phát sinh chi phí chuyển ngữ cảnh.	Tổn chi phí lưu và phục hồi trạng thái tiến trình (PCB).
Ngủ cảnh tối ưu	Dùng khi thời gian thực thi trong CS rất ngắn (thường dùng trong nhân OS, hệ thống đa xử lý).	Dùng khi thời gian trong CS dài hoặc hệ thống đơn vi xử lý.

Bảng 1: So sánh giữa Spinlock và Mutex Non-busy waiting

1.6. Semaphore (Đèn hiệu)

KHÁI NIỆM CỐT LÕI: Định nghĩa Semaphore

Semaphore S là một **biến số nguyên** chỉ có thể truy xuất thông qua 2 thao tác đơn nguyên cơ bản: `wait(S)` (hay $P(S)$) và `signal(S)` (hay $V(S)$).

1.6.1. 1. Phân loại Semaphore

- **Counting Semaphore:** Giá trị là số nguyên không giới hạn, dùng để **quản lý số lượng tài nguyên khả dụng**.
- **Binary Semaphore:** Giá trị chỉ là **0** hoặc **1**, có chức năng tương đương Khóa Mutex.

1.6.2. 2. Hiện thực Semaphore không Busy Waiting

```

1 typedef struct {
2     int value;
3     struct process *list; // Hàng đợi các tiến trình đang ngủ
4 } semaphore;
5
6 void wait(semaphore *S) {
7     S->value--;
8     if (S->value < 0) {
9         // Thêm tiến trình hiện tại vào S->list
10        block(); // Tạm dừng và đưa vào trạng thái ngủ
11    }
12 }
13
14 void signal(semaphore *S) {
15     S->value++;
16     if (S->value <= 0) {
17         // Lấy một tiến trình P ra khỏi S->list
18         wakeup(P); // Đánh thức P và đưa vào Ready queue
19     }
20 }

```

LƯU Ý QUAN TRỌNG ĐI THI: Ý NGHĨA GIÁ TRỊ CỦA SEMAPHORE (CỰC KỲ QUAN TRỌNG)

- Khi $S \rightarrow \text{value} \geq 0$: Biểu thị số tài nguyên còn trống khả dụng (số lần tiến trình có thể gọi `wait()` mà không bị chặn).
- Khi $S \rightarrow \text{value} < 0$: Độ lớn tuyệt đối $|S \rightarrow \text{value}|$ chính là số lượng tiến trình đang bị tắc nghẽn (**blocked**) và nằm trong hàng đợi $S \rightarrow \text{list}$.

1.6.3. 3. Ba ứng dụng kinh điển của Semaphore

1. **Bảo vệ Critical Section:** Khởi tạo $\text{sem} = 1$; Trước CS gọi `wait(sem)`, sau CS gọi `signal(sem)`.
2. **Quy định thứ tự thực thi:** Để đoạn lệnh S_1 (của P_1) luôn chạy trước S_2 (của P_2), khởi tạo $\text{synch} = 0$; P_1 chạy xong S_1 gọi `signal(synch)`; P_2 muốn chạy S_2 phải gọi `wait(synch)`.
3. **Quản lý số lượng tài nguyên hữu hạn:** Khởi tạo $\text{sem} = N$ (với N là tổng số tài nguyên).

1.7. Monitor và Biến điều kiện (Condition Variables)

- **Monitor:** Là kiểu dữ liệu trừu tượng (ADT) cấp cao đóng gói dữ liệu dùng chung và các thủ tục. Điểm then chốt: **Monitor tự động đảm bảo loại trừ tương hỗ (Mutual Exclusion)** – tại một thời điểm chỉ duy nhất 1 tiến trình được thực thi bên trong monitor.
- **Condition Variable (Biến điều kiện):** Cung cấp cơ chế cho tiến trình tạm dừng và chờ đợi bên trong monitor. Khai báo: `condition x;`
 - `x.wait()`: Tiến trình bị block và chuyển vào *condition queue* x , nhường quyền monitor cho tiến trình khác.
 - `x.signal()`: Đánh thức đúng 1 tiến trình đang chờ trên x . Nếu không có tiến trình nào chờ, lệnh này **không có tác dụng** (khác với semaphore sẽ tăng biến đếm).

1.8. Vấn đề Liveness

- **Liveness:** Đặc tính đảm bảo hệ thống duy trì sự tiến triển (tiến trình không bị dừng vô hạn).
- **Deadlock (Tắc nghẽn/Bế tắc):** Hai hay nhiều tiến trình cùng chờ đợi một sự kiện mà sự kiện đó chỉ có thể được tạo ra bởi chính một trong các tiến trình đang chờ.
- **Starvation (Đói tài nguyên):** Tiến trình bị hoãn vô hạn trong hàng đợi, không bao giờ được cấp tài nguyên.
- **Priority Inversion (Nghịch đảo độ ưu tiên):** Hiện tượng tiến trình có độ ưu tiên cao (H) bị chặn do tiến trình ưu tiên thấp (L) đang giữ khóa, trong khi tiến trình ưu tiên trung bình (M) lại chiếm CPU của L .
- **Giải pháp:** Giao thức thừa hưởng ưu tiên (**Priority Inheritance Protocol**): Tạm thời nâng độ ưu tiên của tiến trình L lên bằng mức của H cho đến khi L thả khóa.

1.9. Ba bài toán đồng bộ kinh điển

1.9.1. 1. Bài toán Bounded-Buffer (Producer - Consumer)

Khai báo tài nguyên & Semaphore:

- semaphore mutex = 1; // Đảm bảo loại trừ tương hỗ khi thao tác trên buffer
- semaphore empty = n; // Đếm số vị trí trống còn lại (ban đầu là n)
- semaphore full = 0; // Đếm số vị trí đã có sản phẩm (ban đầu là 0)

```

1 // TIEN TRINH PRODUCER // TIEN TRINH CONSUMER
2 while (true) {         while (true) {
3     /* Tao san pham next_produced */    wait(full);           // Cho co san pham
4     wait(empty);           // Cho co cho trong    wait(mutex);         // Khoa buffer
5     wait(mutex);           // Khoa buffer         // Lay san pham khoi buffer
6     // Them san pham vao buffer    signal(mutex);       // Mo khoa buffer
7     signal(mutex);          // Mo khoa buffer    signal(empty);       // Bao them 1 cho
8     signal(full);           // Bao them 1 sp      trong
9 }                                     }

```

Lưu ý: Bắt buộc gọi `wait(empty/full)` trước `wait(mutex)`. Nếu đảo thứ tự sẽ dẫn đến Deadlock khi bộ đệm đầy/rỗng.

1.9.2. 2. Bài toán Readers - Writers (Biến thể 1: Ưu tiên Readers)

Khai báo dữ liệu chia sẻ:

- semaphore rw_mutex = 1; // Khóa dùng chung giữa Reader và Writer
- semaphore mutex = 1; // Bảo vệ biến đếm read_count
- int read_count = 0; // Đếm số Reader đang đọc

```

1 // TIEN TRINH WRITER // TIEN TRINH READER
2 wait(rw_mutex);       wait(mutex);
3 /* Cap nhat du lieu */    read_count++;
4 signal(rw_mutex);       if (read_count == 1)
5                             wait(rw_mutex); // Reader dau tien
6                             khoa Writer
7                             signal(mutex);
8                             /* Doc du lieu */
9                             wait(mutex);

```

```
9         read_count--;
10         if (read_count == 0)
11             signal(rw_mutex); // Reader cuối cùng
12     }
    }
    signal(mutex);
```

Đánh giá: Biến thể ưu tiên Reader có thể khiến Writer bị đói tài nguyên (Starvation) nếu liên tục có Reader mới đến.

1.9.3. 3. Bài toán Dining-Philosophers (Triết gia ăn tối)

Có 5 triết gia và 5 chiếc đũa (semaphore chopstick[5] = {1, 1, 1, 1, 1}). Triết gia i cần cả đũa trái i và đũa phải $(i + 1) \% 5$.

- **Deadlock:** Nếu cả 5 triết gia cùng nhặt đũa trái đồng thời $\rightarrow$ bế tắc vĩnh viễn.
- **Các giải pháp khắc phục:**
 1. Chỉ cho phép tối đa 4 triết gia cùng ngồi vào bàn.
 2. Chỉ cho phép nhặt đũa khi cả 2 chiếc đũa đều đang rảnh (kiểm tra trong Critical Section).
 3. **Giải pháp bất đối xứng:** Triết gia lẻ nhặt đũa trái trước, đũa phải sau; triết gia chẵn nhặt đũa phải trước, đũa trái sau.

2. CHƯƠNG 7: QUẢN LÝ BỘ NHỚ (MEMORY MANAGEMENT)

2.1. Khái niệm cơ sở và Các kiểu địa chỉ nhớ

- **Mục tiêu quản lý bộ nhớ:** Phân phối, sắp xếp các tiến trình nhằm **tối đa hóa mức độ đa chương (Degree of Multiprogramming)**, bảo vệ không gian nhớ và hỗ trợ chia sẻ dữ liệu.
 - **Phân loại địa chỉ:**
 - **Địa chỉ vật lý (Physical Address):** Vị trí thực tế trên thanh RAM phần cứng.
 - **Địa chỉ luận lý / ảo (Logical / Virtual Address):** Địa chỉ do CPU phát ra khi thực thi chương trình.
 - **Địa chỉ tương đối (Relocatable / Relative Address):** Biểu diễn độ lệch tương đối so với một mốc chuẩn (ví dụ vị trí bắt đầu module).
 - **Địa chỉ tuyệt đối (Absolute Address):** Địa chỉ cố định tương ứng trực tiếp với địa chỉ vật lý.
 - **Linker và Loader:**
 - **Linker:** Liên kết các object modules và thư viện thành một file thực thi duy nhất (*load module*).
 - **Loader:** Nạp *load module* từ đĩa vào bộ nhớ chính để chuẩn bị thực thi.
-

2.2. Chuyển đổi địa chỉ (Address Binding) và Nạp/Liên kết động

2.2.1. 1. Ba thời điểm kết gán địa chỉ (Address Binding)

1. **Thời điểm biên dịch (Compile Time):** Tạo địa chỉ tuyệt đối nếu biết trước vị trí nạp. Bắt buộc phải biên dịch lại nếu địa chỉ nạp thay đổi (ví dụ tệp `.COM` của MS-DOS).
2. **Thời điểm nạp (Load Time):** Trình biên dịch tạo địa chỉ tương đối; Loader sẽ cộng địa chỉ cơ sở khi nạp vào RAM (*tái định vị tĩnh*). Nếu tiến trình bị dời chỗ, phải nạp lại từ đầu.
3. **Thời điểm thực thi (Execution Time):** Ánh xạ địa chỉ được trì hoãn đến lúc lệnh thực sự chạy. **Cần sự hỗ trợ của phần cứng MMU (Memory Management Unit)** với các thanh ghi Base/Limit hoặc Paging. Cho phép tiến trình di chuyển linh hoạt trong RAM.

2.2.2. 2. Nạp động (Dynamic Loading) và Liên kết động (Dynamic Linking)

- **Dynamic Loading:** Một thủ tục/hàm **chỉ được nạp vào RAM khi nó thực sự được gọi đến**. Tối ưu cho các đoạn mã xử lý lỗi hiếm khi xảy ra.
 - **Dynamic Linking:** Việc liên kết với thư viện ngoài được trì hoãn đến khi thực thi. File thực thi chứa một đoạn mã sơ khai (**stub**) để trỏ đến thư viện (`.DLL` trên Windows, `.so` trên Linux).
 - **Ưu điểm vượt trội của Dynamic Linking:** Cho phép cơ chế **chia sẻ mã (Code Sharing)** – một bản sao thư viện duy nhất nạp trong RAM có thể dùng chung cho nhiều tiến trình, tiết kiệm dung lượng RAM và đĩa.
-

2.3. Các mô hình phân vùng bộ nhớ và Hiện tượng phân mảnh

KHÁI NIỆM CỐT LÕI: Phân mảnh bộ nhớ (Fragmentation)

- **Phân mảnh nội (Internal Fragmentation):** Vùng nhớ cấp phát **lớn hơn** vùng nhớ tiến trình yêu cầu. Phần dư thừa nằm bên trong khối đã cấp phát nhưng không được sử dụng. (Thường gặp trong: Fixed Partitioning, Paging).
- **Phân mảnh ngoại (External Fragmentation):** Tổng vùng nhớ trống còn lại **đủ lớn** để thỏa mãn yêu cầu, nhưng các vùng trống bị **phân tán, không liên tục**. (Thường gặp trong: Dynamic Partitioning, Segmentation).
- **Giải pháp cho phân mảnh ngoại:** Kỹ thuật **kết khối (Compaction)** gom các vùng nhớ trống lại thành khối liên tục (chỉ làm được khi địa chỉ được gán tại Execution time).

Bốn chiến lược cấp phát phân vùng động (Dynamic Placement):

1. **First-fit:** Cấp phát khối trống **đầu tiên** đủ lớn (kể từ đầu RAM). Thuật toán **nhANH NHẤT**.
2. **Best-fit:** Cấp phát khối trống **nhỏ nhất** trong số các khối đủ lớn. Tạo ra các **mảnh vụn nhỏ nhất**.
3. **Worst-fit:** Cấp phát khối trống **lớn nhất**. Để lại **phần dư lớn nhất** (để tái sử dụng).
4. **Next-fit:** Tương tự First-fit nhưng bắt đầu tìm kiếm từ **vị trí vừa cấp phát trước đó**.

2.4. Cơ chế Phân trang (Paging) và Bảng trang

- Nguyên lý:
 - Bộ nhớ vật lý chia thành các khối cố định gọi là **Khung trang (Frames)**.
 - Bộ nhớ luận lý chia thành các khối cùng kích thước gọi là **Trang (Pages)**.
 - Kích thước trang luôn là lũy thừa của 2 (2^n , thường từ 4KB đến 16MB).
 - **Ưu điểm:** Cho phép cấp phát bộ nhớ không liên tục; **loại bỏ hoàn toàn phân mảnh ngoại**, chỉ còn phân mảnh nội ở khung trang cuối cùng.
- **Cấu trúc địa chỉ luận lý:** Gồm 2 phần: Số hiệu trang (p) và Độ lệch trong trang (d).
 - Không gian địa chỉ ảo kích thước 2^m , kích thước trang 2^n .
 - Độ lệch (Offset) d : gồm n bits (định vị từ 0 đến $2^n - 1$).
 - Số hiệu trang (Page number) p : gồm $m - n$ bits (định vị từ 0 đến $2^{m-n} - 1$).
 - Bảng trang có tổng cộng 2^{m-n} mục (entries).
- **Cài đặt Bảng trang trong bộ nhớ chính:**
 - **PTBR (Page-Table Base Register):** Thanh ghi trỏ vào địa chỉ bắt đầu của bảng trang.
 - **PTLR (Page-Table Length Register):** Thanh ghi lưu độ dài bảng trang.
 - **Nhược điểm:** Tốn **2 lần truy xuất RAM** cho mỗi thao tác (Lần 1 tra bảng trang, Lần 2 đọc dữ liệu thực tế).

2.5. TLB và Thời gian truy xuất hiệu dụng (EAT)

TLB (Translation Lookaside Buffer) là bộ nhớ đệm phần cứng kết hợp tốc độ cao dùng để lưu các mục bảng trang vừa được truy cập.

LƯU Ý QUAN TRỌNG ĐI THI: CÔNG THỨC TÍNH THỜI GIAN TRUY XUẤT HIỆU DỤNG (EAT)

Gọi:

- α : Tỷ lệ tìm thấy trong TLB (Hit ratio) ($0 \leq \alpha \leq 1$).
- ϵ : Thời gian tìm kiếm trong TLB (Lookup time).
- x : Thời gian truy xuất bộ nhớ chính (Memory access time).

Khi tìm thấy trong TLB (Hit): Thời gian = $\epsilon + x$.

Khi không thấy trong TLB (Miss): Thời gian = $\epsilon + x + x = \epsilon + 2x$.

Công thức tổng quát:

$$\text{EAT} = (\epsilon + x)\alpha + (\epsilon + 2x)(1 - \alpha) = (2 - \alpha)x + \epsilon \quad (1)$$

Nếu đề bài xem thời gian tra cứu TLB rất nhỏ ($\epsilon \approx 0$), công thức thu gọn:

$$\text{EAT} = (2 - \alpha)x \quad (2)$$

2.6. Bảo vệ bộ nhớ và Cơ chế Hoán vị (Swapping)

• Bảo vệ trong phân trang:

- **Protection bits:** Quy định quyền truy xuất (Read-only, Read-Write, Execute-only).
- **Valid / Invalid bit:** Bit valid chứng tỏ trang hợp lệ và đang nằm trong RAM; bit invalid báo hiệu trang không hợp lệ hoặc chưa được nạp vào RAM.

• Cơ chế Hoán vị (Swapping):

- Một tiến trình có thể tạm thời bị đưa ra khỏi RAM và lưu trên bộ nhớ phụ (*swap space / backing store*), sau đó được nạp lại để tiếp tục chạy.
- **Chính sách hoán vị: Round-robin** (hết quantum time thì swap out); **Roll out, roll in** (tiến trình ưu tiên thấp bị swap out nhường chỗ cho tiến trình ưu tiên cao).

3. CHƯƠNG 8: BỘ NHỚ ẢO (VIRTUAL MEMORY)

3.1. Tổng quan về Bộ nhớ ảo và Demand Paging

KHÁI NIỆM CỐT LÕI: Bộ nhớ ảo (Virtual Memory)

Bộ nhớ ảo là kỹ thuật cho phép thực thi một tiến trình mà **không cần nạp toàn bộ tiến trình đó vào bộ nhớ vật lý**. Tách rời hoàn toàn không gian địa chỉ luận lý của người dùng khỏi không gian địa chỉ vật lý của hệ thống.

Ưu điểm chính:

- Cho phép chạy các chương trình có kích thước **lớn hơn nhiều so với dung lượng RAM thực tế**.
- Tăng đáng kể mức độ đa chương (**Degree of Multiprogramming**).
- Giảm thiểu số lượng thao tác I/O cần thiết để nạp hoặc hoán vị tiến trình.

Demand Paging (Phân trang theo yêu cầu):

- Các trang của tiến trình **chỉ được nạp vào RAM khi nó thực sự được CPU truy xuất tới**.
- Sử dụng cờ `valid/invalid` bit trong bảng phân trang để nhận biết trạng thái trang.

3.2. Quy trình xử lý lỗi trang (Page-Fault Service Routine - PFSR)

Lỗi trang (Page Fault) xảy ra khi CPU tham chiếu đến một trang có cờ `invalid` (chưa được nạp vào RAM).

LƯU Ý QUAN TRỌNG ĐI THI: 6 BƯỚC XỬ LÝ LỖI TRANG CỦA HỆ ĐIỀU HÀNH

1. Phần cứng MMU phát hiện bit `invalid` → Phát sinh ngắt nội bộ (**Page-fault trap**) chuyển quyền điều khiển cho OS.
2. OS lưu ngữ cảnh tiến trình, đưa tiến trình vào trạng thái **Blocked (Chờ I/O)**.
3. OS kiểm tra tính hợp lệ của địa chỉ tham chiếu:
 - Nếu địa chỉ bất hợp pháp: Hủy tiến trình (báo lỗi Segmentation Fault).
 - Nếu hợp pháp: Chuẩn bị nạp trang từ đĩa (*backing store*).
4. OS tìm một **khung trang (frame) trống** trong RAM:
 - Nếu có frame trống: Sử dụng frame đó.
 - Nếu không còn frame trống: Dùng **thuật toán thay trang** để chọn một **trang nạn nhân (victim page)**. Nếu trang nạn nhân bị sửa đổi (`dirty bit = 1`), phải ghi nó về đĩa trước khi xóa.
5. OS phát lệnh I/O đọc nội dung trang cần thiết từ đĩa vào khung trang đã chọn.
6. Sau khi I/O hoàn tất (ngắt phần cứng đĩa báo hiệu): Cập nhật bảng trang (gán frame number, sửa cờ thành `valid`), đưa tiến trình về trạng thái **Ready** để thực thi lại chỉ thị gây lỗi trang.

3.3. Các thuật toán thay thế trang (Page Replacement Algorithms)

3.3.1. 1. Thuật toán FIFO (First-In-First-Out)

- **Nguyên tắc:** Thay thế trang **được nạp vào bộ nhớ sớm nhất**.

- **Cài đặt:** Dùng một hàng đợi Queue lưu thứ tự nạp các trang.
- **Nghịch lý Belady (Belady's Anomaly):**
 - Là hiện tượng **số lượng lỗi trang tăng lên** khi **số khung trang được cấp phát tăng lên**.
 - Chuỗi chứng minh kinh điển: 1, 2, 3, 4, 1, 2, 5, 1, 2, 3, 4, 5. Cấp 3 frames xảy ra 9 lỗi trang, nhưng cấp 4 frames lại tăng lên 10 lỗi trang.

3.3.2. 2. Thuật toán Tối ưu (OPT - Optimal)

- **Nguyên tắc:** Thay thế trang sẽ lâu được sử dụng nhất trong tương lai.
- **Đặc điểm:** Cho số lỗi trang **thấp nhất tuyệt đối; không bị Nghịch lý Belady**.
- **Tính khả thi:** **Không thể hiện thực trong thực tế** vì OS không thể dự đoán tương lai; dùng làm **thước đo chuẩn (benchmark)** để so sánh.

3.3.3. 3. Thuật toán LRU (Least Recently Used)

- **Nguyên tắc:** Thay thế trang **đã lâu nhất chưa được sử dụng trong quá khứ**.
- **Đặc điểm:** Xấp xỉ thuật toán OPT dựa trên nguyên lý cục bộ; **không bao giờ bị Nghịch lý Belady** (thuộc nhóm giải thuật Stack).
- **Chi phí:** Cần sự hỗ trợ của phần cứng (bộ đếm thời gian Counter hoặc ngăn xếp Stack liên kết đôi), chi phí quản lý tương đối cao.

3.4. Cấp phát khung trang (Frame Allocation)

- **Cấp phát tĩnh (Fixed Allocation):**
 - **Cấp phát đều (Equal Allocation):** Chia đều m khung trang cho n tiến trình (m/n).
 - **Cấp phát theo tỉ lệ (Proportional Allocation):** Cấp phát dựa trên kích thước s_i của tiến trình P_i :
$$a_i = \frac{s_i}{S} \times m \quad \text{với } S = \sum s_i \quad (3)$$
 - **Cấp phát theo độ ưu tiên:** Cấp nhiều frame hơn cho tiến trình có mức ưu tiên cao hơn.
- **Cấp phát động (Variable Allocation):** Thay đổi số khung trang theo thời gian thực dựa trên tần suất phát sinh lỗi trang (*Page-fault frequency*).

3.5. Hiện tượng Trì trệ (Thrashing) và Mô hình Working-Set

KHÁI NIỆM CỐT LÕI: Hiện tượng Thrashing (Trì trệ toàn hệ thống)

Thrashing là hiện tượng một tiến trình hoặc toàn hệ thống **dành nhiều thời gian cho việc hoán chuyển trang (swap in/out) hơn là thực thi lệnh**.

Hậu quả: Tần suất lỗi trang tăng vọt, **hiệu suất CPU (CPU Utilization) sụt giảm nghiêm trọng**.

Nguyên nhân cốt lõi: Tổng kích thước các tập cục bộ của các tiến trình vượt quá dung lượng RAM:

$$\sum \text{Kích thước Locality} > \text{Dung lượng RAM}$$

Mô hình Working-Set (Working-Set Model):

- Dựa trên **nguyên lý cục bộ (Locality Principle)**: Trong một khoảng thời gian, tiến trình chỉ tập trung truy cập một nhóm trang xác định.
- Định nghĩa tham số cửa sổ trượt Δ (*Working-set window*).
- $WS_i(t)$ là tập hợp các trang được tham chiếu trong Δ lần truy cập gần nhất của tiến trình P_i .
- Kích thước tập làm việc: $WSS_i = |WS_i|$.
- Tổng nhu cầu khung trang của hệ thống: $D = \sum WSS_i$.
- **Chiến lược điều phối của Hệ điều hành:**
 - Nếu $D \leq m$ (với m là tổng số frames của RAM): Hệ thống ổn định.
 - Nếu $D > m$: Nguy cơ Thrashing! OS sẽ **tạm dừng (suspend)** một số tiến trình, swap toàn bộ trang của tiến trình đó ra đĩa để giải phóng frame cho các tiến trình còn lại đạt đủ WSS .

4. CÁC DẠNG BÀI TẬP TÍNH TOÁN ĐI THI (HƯỚNG DẪN CHI TIẾT)

4.1. Dạng 1: Cấp phát bộ nhớ động (First-fit, Best-fit, Worst-fit, Next-fit)

BÀI TẬP VÍ DỤ: Bài tập cấp phát phân vùng

Đề bài: Bộ nhớ chính có các phân vùng trống theo thứ tự: 600K, 500K, 200K, 300K. Các tiến trình yêu cầu cấp phát theo thứ tự: P1 (212K), P2 (417K), P3 (112K), P4 (426K). Hãy xác định phân vùng được cấp phát cho từng tiến trình theo 4 thuật toán và đánh giá thuật toán tối ưu nhất.

Lời giải chi tiết:

1. First-fit (Khối đầu tiên đủ lớn):

- $P_1(212K) \rightarrow$ Cấp vào khối 600K (khối này còn dư $600 - 212 = 388K$).
- $P_2(417K) \rightarrow$ Cấp vào khối 500K (còn dư $500 - 417 = 83K$).
- $P_3(112K) \rightarrow$ Cấp vào phần dư 388K của khối đầu (còn dư $388 - 112 = 276K$).
- $P_4(426K) \rightarrow$ Các khối hiện có: 276K, 83K, 200K, 300K $\rightarrow$ Không có khối nào $\geq 426K \Rightarrow$ **P4 phải chờ**.

2. Best-fit (Khối nhỏ nhất đủ lớn):

- $P_1(212K) \rightarrow$ Chọn khối 300K (dư 88K).
- $P_2(417K) \rightarrow$ Chọn khối 500K (dư 83K).
- $P_3(112K) \rightarrow$ Chọn khối 200K (dư 88K).
- $P_4(426K) \rightarrow$ Chọn khối 600K (dư 174K).
- $\Rightarrow$ **Cả 4 tiến trình đều được cấp phát thành công!**

3. Worst-fit (Khối lớn nhất):

- $P_1(212K) \rightarrow$ Chọn khối lớn nhất 600K (còn dư 388K).
- $P_2(417K) \rightarrow$ Khối lớn nhất hiện tại là 500K $\rightarrow$ cấp vào 500K (dư 83K).
- $P_3(112K) \rightarrow$ Các khối: 388K, 83K, 200K, 300K $\rightarrow$ chọn khối lớn nhất là 388K (còn dư 276K).
- $P_4(426K) \rightarrow$ Các khối: 276K, 83K, 200K, 300K $\rightarrow$ Không khối nào $\geq 426K \Rightarrow$ **P4 phải chờ**.

4. Next-fit (Tiếp tục từ vị trí trước):

- $P_1(212K) \rightarrow$ Cấp vào 600K (dư 388K). Vị trí con trỏ tại khối 600K.
- $P_2(417K) \rightarrow$ Xét tiếp khối 500K $\rightarrow$ cấp vào 500K (dư 83K). Con trỏ tại khối 500K.
- $P_3(112K) \rightarrow$ Xét tiếp khối 200K $\rightarrow$ cấp vào 200K (dư 88K). Con trỏ tại khối 200K.
- $P_4(426K) \rightarrow$ Xét khối 300K (không đủ), quay vòng về 600K (còn dư 388K - không đủ) $\Rightarrow$ **P4 phải chờ**.

Kết luận: Thuật toán **Best-fit** là hiệu quả nhất trong trường hợp này vì thỏa mãn được toàn bộ 4 tiến trình.

4.2. Dạng 2: Phân trang và Tính toán số bit địa chỉ

BÀI TẬP VÍ DỤ: Bài toán số bit trong phân trang

Đề bài 1: Không gian địa chỉ có 12 trang, mỗi trang kích thước 2KB, ánh xạ vào bộ nhớ vật lý có 32 khung trang. Xác định số bit của: (a) Địa chỉ logic, (b) Địa chỉ physical.

Lời giải:

- Kích thước trang = 2 KB = $2 \times 2^{10} = 2^{11}$ bytes $\Rightarrow$ Số bit offset: **d = 11 bits**.
- **Câu (a):** Để đánh số 12 trang (từ 0...11), cần: $\lceil \log_2(12) \rceil = 4$ bits cho số hiệu trang p .
 $\Rightarrow$ Địa chỉ logic gồm: $p + d = 4 + 11 = \mathbf{15}$ bits (tổng không gian nhớ là $12 \times 2\text{KB} = 24\text{KB}$).
- **Câu (b):** Bộ nhớ vật lý có 32 khung trang = 2^5 khung trang $\Rightarrow$ cần **5 bits** cho số hiệu frame f .
 $\Rightarrow$ Địa chỉ vật lý gồm: $f + d = 5 + 11 = \mathbf{16}$ bits (tổng RAM là $32 \times 2\text{KB} = 64\text{KB}$).

BÀI TẬP VÍ DỤ: Bảng trang 2 cấp

Đề bài 2: Máy tính 32-bit địa chỉ ảo, bảng trang 2 cấp: 9 bit cấp 1, 11 bit cấp 2, phần còn lại cho offset. Xác định kích thước 1 trang và số trang tối đa.

Lời giải:

- Số bit offset $d = 32 - (9 + 11) = 32 - 20 = \mathbf{12}$ bits.
- Kích thước 1 trang = 2^{12} bytes = **4 KB**.
- Số lượng trang trong không gian địa chỉ ảo = $2^{32-12} = 2^{20} = \mathbf{1,048,576}$ trang (1M trang).

—

4.3. Dạng 3: Tính thời gian truy xuất hiệu dụng (EAT với TLB)

BÀI TẬP VÍ DỤ: Tính toán EAT

Đề bài: Thời gian truy xuất RAM bình thường $x = 200$ ns.

- Nếu bảng trang lưu trong RAM và không có TLB, thời gian truy xuất là bao nhiêu?
- Nếu có TLB với hit-ratio $\alpha = 75\%$, thời gian tìm trong TLB xem như $\varepsilon = 0$ ns, tính EAT?
- Nếu thời gian tìm kiếm trong TLB là $\varepsilon = 20$ ns và hit-ratio $\alpha = 90\%$, tính EAT?

Lời giải:

- Không có TLB, mỗi thao tác cần 2 lần vào RAM:

$$\text{Thời gian} = 2 \times x = 2 \times 200 \text{ ns} = \mathbf{400 \text{ ns}}$$

- Có TLB với $\varepsilon = 0$, $\alpha = 0.75$:

$$\text{EAT} = \alpha \times x + (1 - \alpha) \times 2x = (0.75 \times 200) + (0.25 \times 400) = 150 + 100 = \mathbf{250 \text{ ns}}$$

- Có TLB với $\varepsilon = 20$ ns, $\alpha = 0.9$:

$$\text{EAT} = (\varepsilon + x)\alpha + (\varepsilon + 2x)(1 - \alpha) = (20 + 200) \times 0.9 + (20 + 400) \times 0.1 = 198 + 42 = \mathbf{240 \text{ ns}}$$

—

4.4. Dạng 4: Mô phỏng thuật toán thay trang (FIFO, OPT, LRU)

BÀI TẬP VÍ DỤ: Mô phỏng thay thế trang

Đề bài: Xét chuỗi tham chiếu bộ nhớ gồm 17 phần tử sau:

1, 2, 3, 4, 2, 1, 5, 6, 2, 1, 2, 3, 7, 6, 3, 2, 1

Hệ thống có **4 khung trang (ban đầu trống)**. Hãy đếm số lỗi trang (Page Fault) đối với: (a) FIFO, (b) OPT, (c) LRU.

1. Thuật toán FIFO (First-In-First-Out - Vào trước ra trước):

- **Nguyên lý:** Trang nào được nạp vào bộ nhớ sớm nhất (thời điểm vào t_{in} nhỏ nhất) sẽ bị thay thế trước khi cần nạp trang mới và bộ nhớ đã đầy.
- **Lưu ý khi làm bài:** Khi xảy ra **Hit** (trang truy xuất đã có sẵn trong khung), thứ tự và thời điểm vào của trang đó **không thay đổi**.

Chuỗi	1	2	3	4	2	1	5	6	2	1	2	3	7	6	3	2	1
Khung 1	1	1	1	1	1	1	5	5	5	5	5	3	3	3	3	3	1
Khung 2		2	2	2	2	2	2	6	6	6	6	6	7	7	7	7	7
Khung 3			3	3	3	3	3	2	2	2	2	2	2	6	6	6	6
Khung 4				4	4	4	4	4	4	1	1	1	1	1	1	2	2
Fault?	*	*	*	*			*	*	*	*		*	*	*		*	*

Diễn giải các bước thay thế quan trọng (FIFO):

- Bước 1–4: Lần lượt nạp 1, 2, 3, 4 vào 4 khung trống $\Rightarrow$ 4 lỗi trang.
- Bước 5 (trang 2) & Bước 6 (trang 1): Đã có sẵn trong RAM $\Rightarrow$ Hit.
- Bước 7 (trang 5): Thay trang 1 ở Khung 1 ($t_{in} = 1$ sớm nhất). Khung: {5, 2, 3, 4}.
- Bước 8 (trang 6): Thay trang 2 ở Khung 2 ($t_{in} = 2$ sớm nhất). Khung: {5, 6, 3, 4}.
- Bước 9 (trang 2): Thay trang 3 ở Khung 3 ($t_{in} = 3$ sớm nhất). Khung: {5, 6, 2, 4}.
- Bước 10 (trang 1): Thay trang 4 ở Khung 4 ($t_{in} = 4$ sớm nhất). Khung: {5, 6, 2, 1}.
- Bước 11 (trang 2): Hit (đã có ở Khung 3).
- Bước 12 (trang 3): Thay trang 5 ở Khung 1 ($t_{in} = 7$ sớm nhất). Khung: {3, 6, 2, 1}.
- Bước 13 (trang 7): Thay trang 6 ở Khung 2 ($t_{in} = 8$ sớm nhất). Khung: {3, 7, 2, 1}.
- Bước 14 (trang 6): Thay trang 2 ở Khung 3 ($t_{in} = 9$ sớm nhất). Khung: {3, 7, 6, 1}.
- Bước 15 (trang 3): Hit (đã có ở Khung 1).
- Bước 16 (trang 2): Thay trang 1 ở Khung 4 ($t_{in} = 10$ sớm nhất). Khung: {3, 7, 6, 2}.
- Bước 17 (trang 1): Thay trang 3 ở Khung 1 ($t_{in} = 12$ sớm nhất). Khung: {1, 7, 6, 2}.

$\Rightarrow$ **Tổng số lỗi trang (FIFO): 13 lỗi trang.**

2. Thuật toán OPT (Optimal Page Replacement - Thay thế tối ưu):

- **Nguyên lý:** Nhìn về **tương lai** (từ sau vị trí hiện tại nhìn sang phải). Thay thế trang nào **lâu được dùng lại nhất** hoặc **không bao giờ được dùng lại nữa** (∞).
- **Quy tắc ưu tiên:** Trang nào không bao giờ xuất hiện lại trong tương lai bắt buộc phải ưu tiên thay thế trước!

Chuỗi	1	2	3	4	2	1	5	6	2	1	2	3	7	6	3	2	1
Khung 1	1	1	1	1	1	1	1	1	1	1	1	1	7	7	7	7	1
Khung 2		2	2	2	2	2	2	2	2	2	2	2	2	2	2	2	2
Khung 3			3	3	3	3	3	3	3	3	3	3	3	3	3	3	3
Khung 4				4	4	4	5	6	6	6	6	6	6	6	6	6	6
Fault?	*	*	*	*			*	*					*				*

Hướng dẫn giải chi tiết từng cột (Cột 1 đến Cột 17):

Cột 1: Trang 1: Khung 1 còn trống $\rightarrow$ Nạp 1 vào Khung 1 $\Rightarrow$ **Lỗi trang (*)**. Khung: {1, -, -, -}.

Cột 2: Trang 2: Khung 2 còn trống $\rightarrow$ Nạp 2 vào Khung 2 $\Rightarrow$ **Lỗi trang (*)**. Khung: {1, 2, -, -}.

Cột 3: Trang 3: Khung 3 còn trống $\rightarrow$ Nạp 3 vào Khung 3 $\Rightarrow$ **Lỗi trang (*)**. Khung: {1, 2, 3, -}.

Cột 4: Trang 4: Khung 4 còn trống $\rightarrow$ Nạp 4 vào Khung 4 $\Rightarrow$ **Lỗi trang (*)**. Khung: {1, 2, 3, 4} (đầy 4 khung).

Cột 5: Trang 2: Đã có sẵn ở Khung 2 $\Rightarrow$ **Hit**. Khung giữ nguyên: {1, 2, 3, 4}.

Cột 6: Trang 1: Đã có sẵn ở Khung 1 $\Rightarrow$ **Hit**. Khung giữ nguyên: {1,2,3,4}.

Cột 7: Trang 5: Chưa có $\rightarrow$ **Lỗi trang (*)**. Khung đầy {1,2,3,4}, soi tương lai phía sau (Cột 8–17: 6, 2, 1, 2, 3, 7, 6, 3, 2, 1):

- Trang 2 xuất hiện ở Cột 9; Trang 1 ở Cột 10; Trang 3 ở Cột 12.
- Trang 4 **không bao giờ xuất hiện lại nữa** (∞) $\Rightarrow$ Chọn trang 4 làm victim.

Thay trang 4 bằng 5 ở Khung 4. Khung sau bước 7: {1,2,3,5}.

Cột 8: Trang 6: Chưa có $\rightarrow$ **Lỗi trang (*)**. Khung đầy {1,2,3,5}, soi tương lai phía sau (Cột 9–17: 2, 1, 2, 3, 7, 6, 3, 2, 1):

- Trang 2 ở Cột 9; Trang 1 ở Cột 10; Trang 3 ở Cột 12.
- Trang 5 **không bao giờ xuất hiện lại nữa** (∞) $\Rightarrow$ Chọn trang 5 làm victim.

Thay trang 5 bằng 6 ở Khung 4. Khung sau bước 8: {1,2,3,6}.

Cột 9: Trang 2: Đã có sẵn ở Khung 2 $\Rightarrow$ **Hit**. Khung giữ nguyên: {1,2,3,6}.

Cột 10: Trang 1: Đã có sẵn ở Khung 1 $\Rightarrow$ **Hit**. Khung giữ nguyên: {1,2,3,6}.

Cột 11: Trang 2: Đã có sẵn ở Khung 2 $\Rightarrow$ **Hit**. Khung giữ nguyên: {1,2,3,6}.

Cột 12: Trang 3: Đã có sẵn ở Khung 3 $\Rightarrow$ **Hit** (nhờ bước 7 đã giữ lại trang 3). Khung giữ nguyên: {1,2,3,6}.

Cột 13: Trang 7: Chưa có $\rightarrow$ **Lỗi trang (*)**. Khung đầy {1,2,3,6}, soi tương lai phía sau (Cột 14–17: 6, 3, 2, 1):

- Trang 6 ở Cột 14 (cách 1); Trang 3 ở Cột 15 (cách 2); Trang 2 ở Cột 16 (cách 3).
- Trang 1 ở Cột 17 (**xa nhất trong 4 trang**) $\Rightarrow$ Chọn trang 1 làm victim.

Thay trang 1 bằng 7 ở Khung 1. Khung sau bước 13: {7,2,3,6}.

Cột 14: Trang 6: Đã có sẵn ở Khung 4 $\Rightarrow$ **Hit**. Khung giữ nguyên: {7,2,3,6}.

Cột 15: Trang 3: Đã có sẵn ở Khung 3 $\Rightarrow$ **Hit**. Khung giữ nguyên: {7,2,3,6}.

Cột 16: Trang 2: Đã có sẵn ở Khung 2 $\Rightarrow$ **Hit**. Khung giữ nguyên: {7,2,3,6}.

Cột 17: Trang 1: Chưa có $\rightarrow$ **Lỗi trang (*)**. Tương lai không còn trang nào nữa, trang 7 không dùng nữa $\rightarrow$ Thay trang 7 bằng 1 ở Khung 1. Khung sau bước 17: {1,2,3,6}.

$\Rightarrow$ **Tổng số lỗi trang (OPT): 8 lỗi trang.**

3. Thuật toán LRU (Least Recently Used - Lâu nhất chưa sử dụng):

- **Nguyên lý:** Nhìn về **quá khứ** (từ vị trí hiện tại nhìn ngược về bên trái). Thay thế trang có lần truy xuất gần nhất cách xa hiện tại nhất.
- **Mẹo làm bài nhanh:** Dò ngược chuỗi tham chiếu từ phải qua trái. 3 trang đầu tiên bắt gặp sẽ được giữ lại, trang thứ 4 gặp sau cùng trong khung chính là nạn nhân (victim).

Chuỗi	1	2	3	4	2	1	5	6	2	1	2	3	7	6	3	2	1
Khung 1	1	1	1	1	1	1	1	1	1	1	1	1	1	6	6	6	6
Khung 2	2	2	2	2	2	2	2	2	2	2	2	2	2	2	2	2	2
Khung 3			3	3	3	3	5	5	5	5	5	3	3	3	3	3	3
Khung 4				4	4	4	4	6	6	6	6	6	7	7	7	7	1
Fault?	*	*	*	*			*	*				*	*	*			*

Hướng dẫn giải chi tiết từng cột (Cột 1 đến Cột 17):

Cột 1: Trang 1: Khung 1 còn trống $\rightarrow$ Nạp 1 vào Khung 1 $\Rightarrow$ **Lỗi trang (*)**. Khung: $\{1, -, -, -\}$. Thứ tự dùng gần $\rightarrow$ xa: (1).

Cột 2: Trang 2: Khung 2 còn trống $\rightarrow$ Nạp 2 vào Khung 2 $\Rightarrow$ **Lỗi trang (*)**. Khung: $\{1, 2, -, -\}$. Thứ tự dùng: (2, 1).

Cột 3: Trang 3: Khung 3 còn trống $\rightarrow$ Nạp 3 vào Khung 3 $\Rightarrow$ **Lỗi trang (*)**. Khung: $\{1, 2, 3, -\}$. Thứ tự dùng: (3, 2, 1).

Cột 4: Trang 4: Khung 4 còn trống $\rightarrow$ Nạp 4 vào Khung 4 $\Rightarrow$ **Lỗi trang (*)**. Khung: $\{1, 2, 3, 4\}$ (đầy 4 khung). Thứ tự dùng: (4, 3, 2, 1).

Cột 5: Trang 2: Đã có sẵn ở Khung 2 $\Rightarrow$ **Hit**. Khung giữ nguyên: $\{1, 2, 3, 4\}$. Cập nhật thứ tự dùng (2 lên đầu): (2, 4, 3, 1).

Cột 6: Trang 1: Đã có sẵn ở Khung 1 $\Rightarrow$ **Hit**. Khung giữ nguyên: $\{1, 2, 3, 4\}$. Cập nhật thứ tự dùng (1 lên đầu): (1, 2, 4, 3).

Cột 7: Trang 5: Chưa có $\rightarrow$ **Lỗi trang (*)**. Khung đầy $\{1, 2, 3, 4\}$, dò ngược về quá khứ từ Cột 6 sang trái:

- Cột 6 gặp 1; Cột 5 gặp 2; Cột 4 gặp 4; Cột 3 mới gặp 3 (**xa nhất về bên trái**).
- $\Rightarrow$ Trang 3 lâu nhất chưa dùng $\Rightarrow$ Chọn trang 3 làm victim.

Thay trang 3 bằng 5 ở Khung 3. Khung sau bước 7: $\{1, 2, 5, 4\}$. Thứ tự dùng: (5, 1, 2, 4).

Cột 8: Trang 6: Chưa có $\rightarrow$ **Lỗi trang (*)**. Khung đầy $\{1, 2, 5, 4\}$, dò ngược về quá khứ từ Cột 7 sang trái:

- Cột 7 gặp 5; Cột 6 gặp 1; Cột 5 gặp 2; Cột 4 mới gặp 4 (**xa nhất về bên trái**).
- $\Rightarrow$ Trang 4 lâu nhất chưa dùng $\Rightarrow$ Chọn trang 4 làm victim.

Thay trang 4 bằng 6 ở Khung 4. Khung sau bước 8: $\{1, 2, 5, 6\}$. Thứ tự dùng: (6, 5, 1, 2).

Cột 9: Trang 2: Đã có sẵn ở Khung 2 $\Rightarrow$ **Hit**. Khung giữ nguyên: $\{1, 2, 5, 6\}$. Thứ tự dùng: (2, 6, 5, 1).

Cột 10: Trang 1: Đã có sẵn ở Khung 1 $\Rightarrow$ **Hit**. Khung giữ nguyên: $\{1, 2, 5, 6\}$. Thứ tự dùng: (1, 2, 6, 5).

Cột 11: Trang 2: Đã có sẵn ở Khung 2 $\Rightarrow$ **Hit**. Khung giữ nguyên: $\{1, 2, 5, 6\}$. Thứ tự dùng: (2, 1, 6, 5).

Cột 12: Trang 3: Chưa có $\rightarrow$ **Lỗi trang (*)**. Khung đầy $\{1, 2, 5, 6\}$, dò ngược về quá khứ từ Cột 11 sang trái:

- Cột 11 gặp 2; Cột 10 gặp 1; Cột 8 gặp 6; Cột 7 mới gặp 5 (**xa nhất về bên trái**).
- $\Rightarrow$ Trang 5 lâu nhất chưa dùng $\Rightarrow$ Chọn trang 5 làm victim.

Thay trang 5 bằng 3 ở Khung 3. Khung sau bước 12: $\{1, 2, 3, 6\}$. Thứ tự dùng: (3, 2, 1, 6).

Cột 13: Trang 7: Chưa có $\rightarrow$ **Lỗi trang (*)**. Khung đầy $\{1, 2, 3, 6\}$, dò ngược về quá khứ từ Cột 12 sang trái:

- Cột 12 gặp 3; Cột 11 gặp 2; Cột 10 gặp 1; Cột 8 mới gặp 6 (**xa nhất về bên trái**).
- $\Rightarrow$ Trang 6 lâu nhất chưa dùng $\Rightarrow$ Chọn trang 6 làm victim.

Thay trang 6 bằng 7 ở Khung 4. Khung sau bước 13: $\{1, 2, 3, 7\}$. Thứ tự dùng: (7, 3, 2, 1).

Cột 14: Trang 6: Chưa có $\rightarrow$ **Lỗi trang (*)**. Khung đầy $\{1, 2, 3, 7\}$, dò ngược về quá khứ từ Cột 13 sang trái:

- Cột 13 gặp 7; Cột 12 gặp 3; Cột 11 gặp 2; Cột 10 mới gặp 1 (**xa nhất về bên trái**).
- $\Rightarrow$ Trang 1 lâu nhất chưa dùng $\Rightarrow$ Chọn trang 1 làm victim.

Thay trang 1 bằng 6 ở Khung 1. Khung sau bước 14: $\{6, 2, 3, 7\}$. Thứ tự dùng: (6, 7, 3, 2).

Cột 15: Trang 3: Đã có sẵn ở Khung 3 $\Rightarrow$ **Hit**. Khung giữ nguyên: $\{6, 2, 3, 7\}$. Thứ tự dùng: (3, 6, 7, 2).

Cột 16: Trang 2: Đã có sẵn ở Khung 2 $\Rightarrow$ **Hit**. Khung giữ nguyên: $\{6, 2, 3, 7\}$. Thứ tự dùng: (2, 3, 6, 7).

Cột 17: Trang 1: Chưa có $\rightarrow$ **Lỗi trang (*)**. Khung đầy $\{6, 2, 3, 7\}$, dò ngược về quá khứ từ Cột 16 sang trái:

- Cột 16 gấp 2; Cột 15 gấp 3; Cột 14 gấp 6; Cột 13 mới gấp 7 (**xa nhất về bên trái**).
- $\Rightarrow$ Trang 7 lâu nhất chưa dùng $\Rightarrow$ Chọn trang 7 làm victim.

Thay trang 7 bằng 1 ở Khung 4. Khung sau bước 17: $\{6, 2, 3, 1\}$.

$\Rightarrow$ **Tổng số lỗi trang (LRU): 10 lỗi trang.**

LƯU Ý QUAN TRỌNG ĐI THI: So sánh phản xạ nhanh khi làm bài thi

- **Số lỗi trang:** $\text{OPT}(8) \leq \text{LRU}(10) \leq \text{FIFO}(13)$. Thuật toán OPT luôn cho số lỗi trang ít nhất.
- **Nghịch lý Belady:** Chỉ có **FIFO** có thể bị nghịch lý Belady (tăng số khung trang nhưng số lỗi trang lại tăng). **OPT** và **LRU** thuộc nhóm *Stack Algorithms* nên không bao giờ bị nghịch lý này.

4.5. Dạng 5: Cấp phát khung trang theo tỉ lệ (Proportional Allocation)

BÀI TẬP VÍ DỤ: Cấp phát theo tỉ lệ

Đề bài: Hệ thống có $m = 64$ frames khả dụng. Có 2 tiến trình: P_1 có kích thước $s_1 = 10$ trang, P_2 có kích thước $s_2 = 127$ trang. Tính số frame cấp cho mỗi tiến trình.

Lời giải:

- Tổng kích thước: $S = s_1 + s_2 = 10 + 127 = 137$ trang.
- Số frame cho P_1 : $a_1 = \frac{10}{137} \times 64 \approx 4.67 \Rightarrow \mathbf{5}$ frames.
- Số frame cho P_2 : $a_2 = \frac{127}{137} \times 64 \approx 59.33 \Rightarrow \mathbf{59}$ frames.

5. BẢNG SO SÁNH TỔNG HỢP VÀ BỘ CÂU HỎI PHẢN XẠ NHANH

5.1. Bảng phân biệt các khái niệm then chốt

Tiêu chí	Khái niệm A	Khái niệm B
Phân mảnh	Phân mảnh nội: Vùng nhớ dư thừa nằm bên trong khối đã cấp (Fixed partition, Paging).	Phân mảnh ngoại: Vùng nhớ trống nằm rải rác bên ngoài , tổng đủ nhưng không liên tục (Dynamic partition).
Loading vs Linking	Dynamic Loading: Chỉ nạp thủ tục vào RAM khi được gọi (tiết kiệm bộ nhớ cho code ít dùng).	Dynamic Linking: Dùng Stub liên kết thư viện lúc chạy, nhiều process dùng chung một bản copy trên RAM .
Mutex vs Semaphore	Mutex Lock: Có tính chất Ownership (tiền trình nào khóa thì chính nó phải mở khóa).	Binary Semaphore: Không có ownership (tiền trình này gọi wait, tiền trình khác có thể gọi signal).
Paging vs Swapping	Paging: Di chuyển từng trang nhớ đơn lẻ giữa RAM và đĩa (bộ nhớ ảo).	Swapping: Di chuyển toàn bộ tiến trình ra/vào bộ nhớ phụ.
FIFO vs OPT vs LRU	FIFO: Dễ cài đặt nhất, bị Nghịch lý Belady .	OPT: Tối ưu nhất, không bị Belady, không khả thi thực tế . LRU: Tiệm cận OPT, không bị Belady, cần phần cứng.

Bảng 2: Bảng so sánh tổng hợp các cặp khái niệm dễ nhầm lẫn

5.2. Bộ câu hỏi phản xạ nhanh lý thuyết đi thi

- Hỏi:** Tại sao cơ chế tắt ngắt (*disable interrupts*) không giải quyết được vấn đề đồng bộ trên hệ thống đa xử lý (Multiprocessor)?
→ **Đáp:** Vì lệnh tắt ngắt chỉ có hiệu lực trên chính CPU đang chạy lệnh đó. Các CPU khác trong hệ thống vẫn tiếp tục chạy và truy cập vào bộ nhớ chia sẻ bình thường.
- Hỏi:** Khi Semaphore S có giá trị âm ($S \rightarrow \text{value} = -4$), con số đó có ý nghĩa gì?
→ **Đáp:** Có chính xác **4 tiến trình đang bị tắc nghẽn (blocked)** và đang nằm chờ trong hàng đợi của Semaphore đó.
- Hỏi:** Nghịch lý Belady (*Belady's Anomaly*) là gì? Thường xảy ra ở thuật toán thay trang nào?
→ **Đáp:** Là hiện tượng **tỷ lệ lỗi trang tăng lên khi số khung trang cấp phát cho tiến trình tăng lên**. Thuật toán điển hình mắc phải là **FIFO**.
- Hỏi:** Tại sao thuật toán thay trang OPT không thể cài đặt trong các hệ điều hành thực tế?
→ **Đáp:** Vì hệ điều hành không thể biết trước được chuỗi tham chiếu bộ nhớ trong tương lai của một tiến trình.
- Hỏi:** Hiện tượng Thrashing xảy ra khi nào và hệ điều hành ngăn chặn nó bằng cách nào?
→ **Đáp:** Xảy ra khi tổng kích thước các tập làm việc vượt quá dung lượng RAM thực tế ($D > m$). Hệ điều hành ngăn chặn bằng cách **tạm dừng (suspend)** một số tiến trình để thu hồi frame, đảm bảo các tiến trình còn lại có đủ số khung trang cho working-set của chúng.
- Hỏi:** Trong bài toán Bounded-Buffer, tại sao Producer phải gọi `wait(empty)` trước `wait(mutex)`?
→ **Đáp:** Nếu gọi `wait(mutex)` trước, khi buffer bị đầy, Producer sẽ giữ khóa mutex rồi bị kẹt ở `wait(empty)`. Lúc này Consumer muốn vào lấy hàng để giải phóng chỗ trống cũng không thể lấy được khóa mutex → Hệ thống rơi vào **Deadlock**.